DESTINO3 · DOCUMENTACIÓN TÉCNICA

Guía de la base de datos

Estado actual (versión as-is) · PostgreSQL · 7 tablas

7	7	2	1
tablas	llaves foráneas	tablas raíz	tabla puente N:M

Índice

Visión general 2

1. Diagrama Entidad-Relación 3

2. Mapa técnico de conexiones y restricciones 4

2.1 Relaciones entre tablas (7 llaves foráneas) 4

2.2 Ficha de cada tabla: llave primaria y restricciones propias 4

2.3 Cómo se conectan las dos raíces con el resto del grafo 4

2.4 Propagación de ON DELETE CASCADE 5

2.5 Particularidades del esquema tal como está hoy 5

3. Flujo de datos del negocio 6

3.1 Fase 1 · Registro de entidades raíz 6

3.2 Fase 2 · Búsqueda 7

3.3 Fase 3 · Registro del catálogo del destino 8

3.4 Fase 4 · Generación de recomendaciones (Top 3) 9

Anexo · Código Mermaid 10

A. Diagrama ER 10

B. Diagrama de flujo (completo) 11

Cómo navegar este PDF

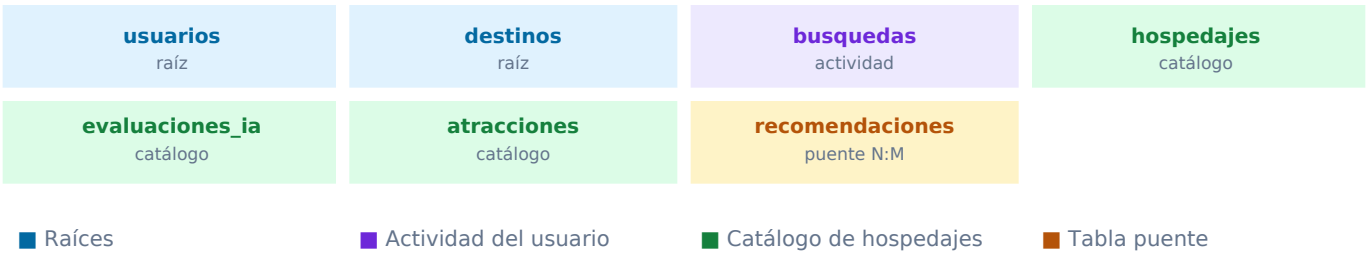
- Cada línea del índice es un enlace; también tienes el panel de **marcadores** de tu visor.
- Los nombres de tabla en **color** saltan a su ficha (sección 2.2).
- El pie de página ↑ Volver al índice te devuelve aquí desde cualquier página.
- El código Mermaid original está en el Anexo, listo para copiar.

Visión general

El esquema tiene dos raíces independientes, **usuarios** y **destinos**, y todo lo demás cuelga de ellas. Se lee mejor como dos ramas que se encuentran en una tabla:

- **Rama de catálogo:** destinos → hospedajes → (evaluaciones_ia, atracciones).
- **Rama de actividad:** usuarios + destinos → busquedas → recomendaciones.
- **Convergencia:** recomendaciones apunta a la búsqueda y al hospedaje, y une ambas ramas.

Mapa de tablas (haz clic en una tabla para ir a su ficha)



1. Diagrama Entidad-Relación

Diagrama completo de las 7 tablas, con atributos, tipos, llaves (PK, FK, UK) y la notación de cardinalidad de Mermaid.

- **||** exactamente uno · **o|** o **|o** cero o uno · **o{** cero o muchos.
- Mermaid no admite comas dentro del tipo, por eso DECIMAL(10,2) aparece como `DECIMAL_10_2`.
- Mermaid no dibuja el índice único funcional de destinos; está descrito en la sección 2.2.
- recomendaciones es la tabla puente (N:M) entre busquedas y hospedajes, con posicion como atributo propio.

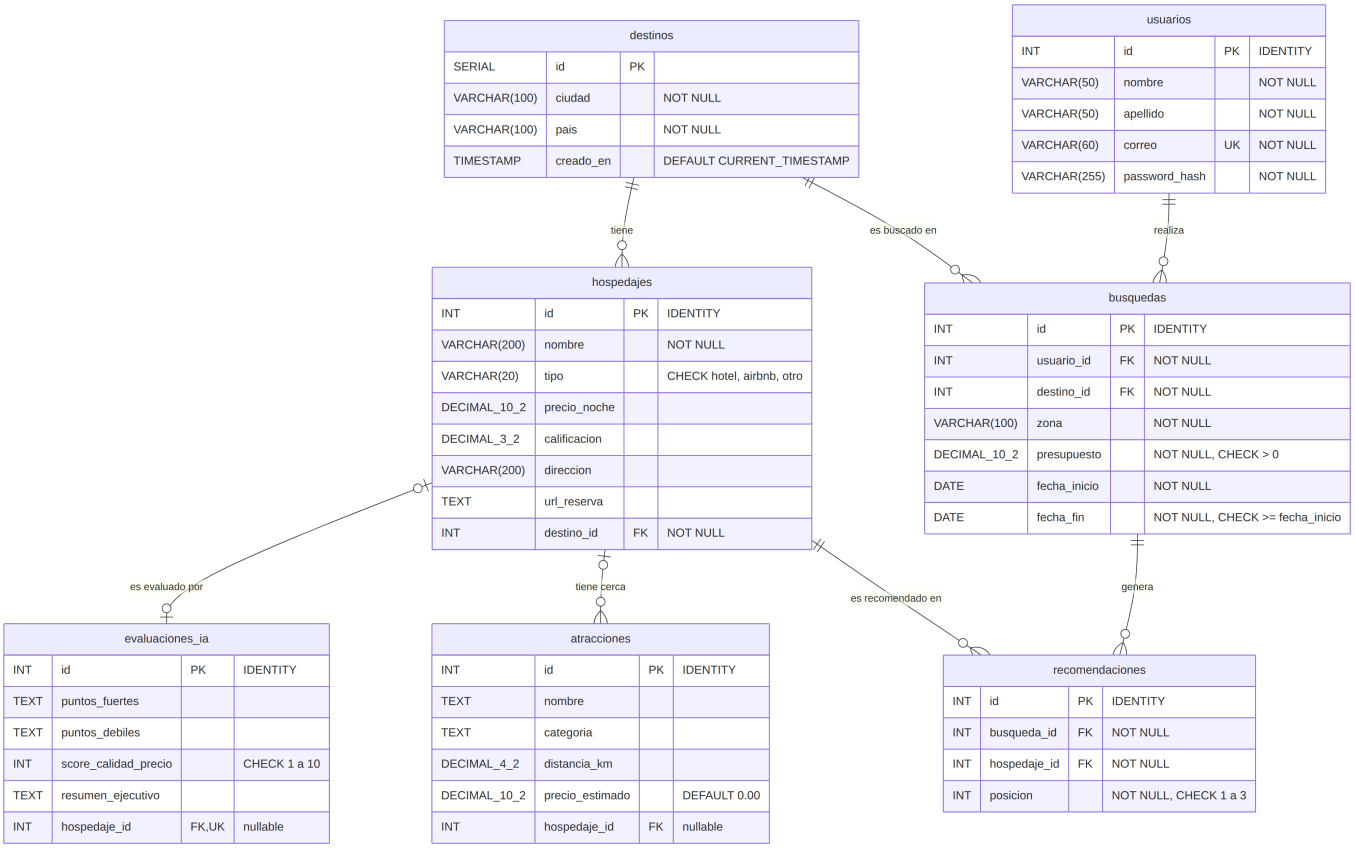


Figura 1 · Diagrama ER del esquema actual

Ir a la ficha de: usuarios · destinos · busquedas · hospedajes · evaluaciones_ia · atracciones · recomendaciones

2. Mapa técnico de conexiones y restricciones

2.1 Relaciones entre tablas (7 llaves foráneas)

#	Tabla origen (hija)	Tabla destino (padre)	FK	PK referenciada	Cardinalidad	Comportamiento
1	**hospedajes**	**destinos**	destino_id (NOT NULL)	destinos.id	1:N	ON DELETE CASCADE · índice idx_hospedajes_destino_id
2	**evaluaciones_ia**	**hospedajes**	hospedaje_id (nullable)	hospedajes.id	1:1 opcional (0..1)	UNIQUE sobre hospedaje_id · ON DELETE CASCADE
3	**atracciones**	**hospedajes**	hospedaje_id (nullable)	hospedajes.id	1:N	ON DELETE CASCADE
4	**busquedas**	**usuarios**	usuario_id (NOT NULL)	usuarios.id	1:N	ON DELETE CASCADE
5	**busquedas**	**destinos**	destino_id (NOT NULL)	destinos.id	1:N	ON DELETE CASCADE
6	**recomendaciones**	**busquedas**	busqueda_id (NOT NULL)	busquedas.id	1:N (máx. 3 por búsqueda)	ON DELETE CASCADE · UNIQUE (busqueda_id, posicion) · UNIQUE (busqueda_id, hospedaje_id)
7	**recomendaciones**	**hospedajes**	hospedaje_id (NOT NULL)	hospedajes.id	1:N	ON DELETE CASCADE

Las filas 6 y 7 juntas forman una relación **N:M** entre busquedas y hospedajes; recomendaciones es la tabla intermedia.

2.2 Ficha de cada tabla: llave primaria y restricciones propias

Tabla	PK	Restricciones locales
destinos	id (SERIAL)	Índice único funcional `idx_destinos_ciudad_pais_normalizado` sobre LOWER(TRIM(ciudad)) y LOWER(TRIM(pais)): impide duplicar la misma ciudad y país aunque cambien mayúsculas o espacios.
hospedajes	id (IDENTITY)	CHECK tipo IN ('hotel', 'airbnb', 'otro')
evaluaciones_ia	id (IDENTITY)	CHECK score_calidad_precio BETWEEN 1 AND 10 · UNIQUE hospedaje_id
atracciones	id (IDENTITY)	precio_estimado DEFAULT 0.00
usuarios	id (IDENTITY)	UNIQUE correo · nombre, apellido y password_hash son NOT NULL
busquedas	id (IDENTITY)	`chk_busqueda_presupuesto`: presupuesto > 0 · `chk_busqueda_fechas`: fecha_fin >= fecha_inicio
recomendaciones	id (IDENTITY)	`chk_recomendacion_posicion`: posicion BETWEEN 1 AND 3 · `uq_recomendacion_busqueda_posicion` · `uq_recomendacion_busqueda_hospedaje`

2.3 Cómo se conectan las dos raíces con el resto del grafo

destinos (raíz geográfica) alimenta dos ramas. En la **rama de catálogo**, destinos → hospedajes → evaluaciones_ia y atracciones: un destino agrupa los hospedajes disponibles, cada hospedaje tiene como máximo una evaluación de IA y puede tener varias atracciones cercanas. En la **rama de actividad**, destinos → busquedas: cada búsqueda apunta a un destino.

usuarios (raíz de identidad) se conecta a una sola tabla, busquedas. De ahí en adelante, el usuario se alcanza solo a través de sus búsquedas.

Punto de convergencia: recomendaciones une las dos ramas. Recibe busqueda_id desde la rama de actividad y hospedaje_id desde la rama de catálogo; así, un mismo hospedaje puede aparecer en búsquedas de distintos usuarios.

2.4 Propagación de ON DELETE CASCADE

Las siete FK son CASCADE, así que el borrado se propaga hacia abajo:

Al borrar…	Se elimina…
un destino	Sus hospedajes y sus busquedas; en cadena, evaluaciones_ia, atracciones y recomendaciones. Las recomendaciones pueden caer por dos caminos: vía hospedajes y vía busquedas.
un usuario	Sus busquedas y, con ellas, sus recomendaciones. No toca el catálogo.
un hospedaje	Su evaluacion_ia, sus atracciones y las recomendaciones donde aparecía.
una búsqueda	Únicamente sus recomendaciones.

2.5 Particularidades del esquema tal como está hoy

- `evaluaciones_ia.hospedaje_id` y `atracciones.hospedaje_id` son nullable: a nivel de esquema, ambas tablas admiten filas sin hospedaje asociado.
- atracciones no tiene vínculo directo con destinos; llega a él solo a través de hospedajes.
- recomendaciones permite como máximo 3 filas por búsqueda (posicion 1 a 3 con UNIQUE). El esquema no fuerza un mínimo de 3.
- El esquema no verifica que el hospedaje_id de una recomendación pertenezca al mismo destino_id de la búsqueda; esa coherencia depende de la lógica de la aplicación.
- destinos.id usa SERIAL; las demás tablas usan GENERATED BY DEFAULT AS IDENTITY.

3. Flujo de datos del negocio

Ciclo de vida de los datos en cuatro fases. Primero la vista general; después, el detalle de cada fase con sus validaciones. Los nodos con forma de cilindro son tablas y los rojos son rechazos por restricción.

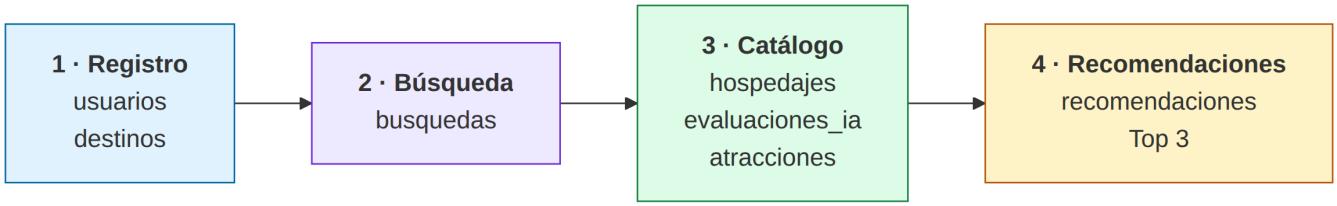


Figura 2 · Vista general: de los datos raíz a las recomendaciones

Lectura: las fases 1 y 2 crean la identidad y la intención (quién busca, dónde, con qué presupuesto y en qué fechas). La fase 3 llena el catálogo; en el esquema, hospedajes depende solo de destinos, no de busquedas. La fase 4 cierra el ciclo juntando la búsqueda con los hospedajes ya evaluados.

3.1 Fase 1 · Registro de entidades raíz

Dos flujos independientes: el usuario se registra con correo único, y el destino se crea solo si no existe ya (comparación normalizada de ciudad y país).

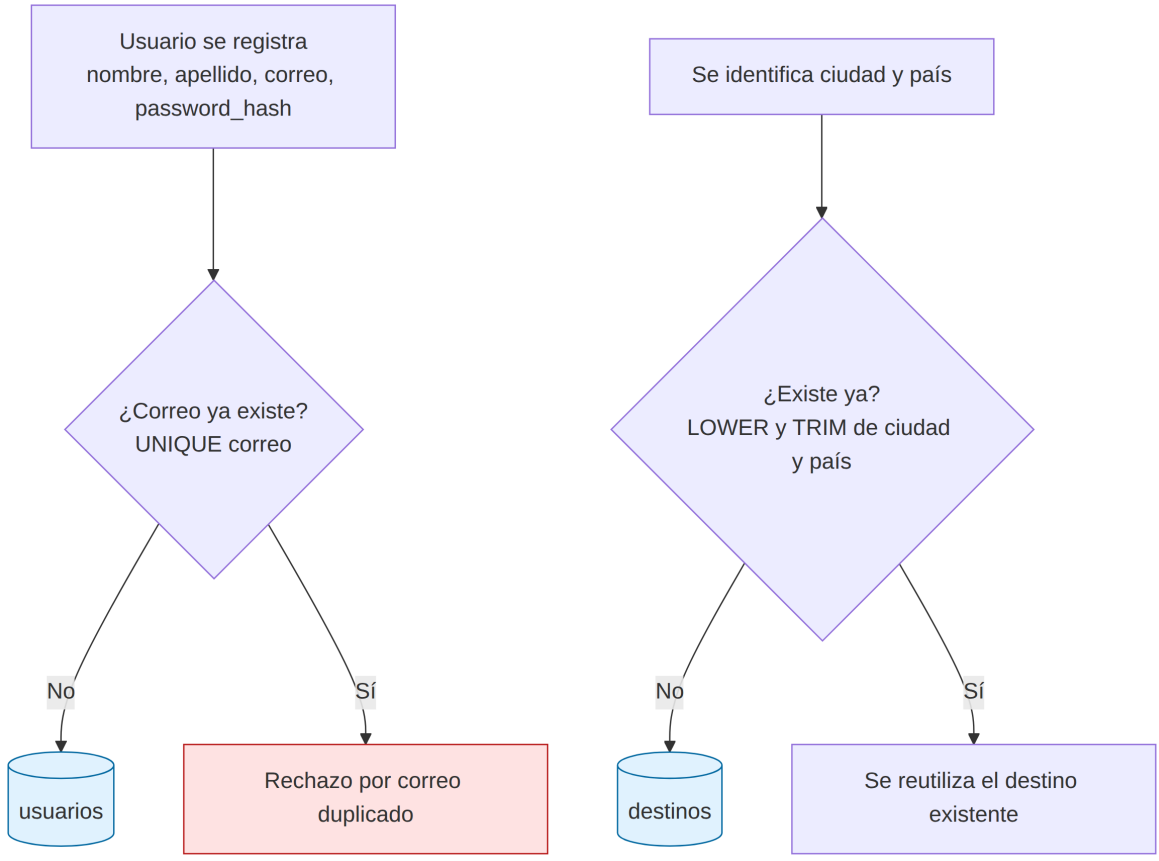


Figura 3 · Fase 1: usuarios y destinos

3.2 Fase 2 · Búsqueda

El usuario registrado define zona, presupuesto y fechas sobre un destino existente. Dos CHECK validan el presupuesto y el orden de las fechas.

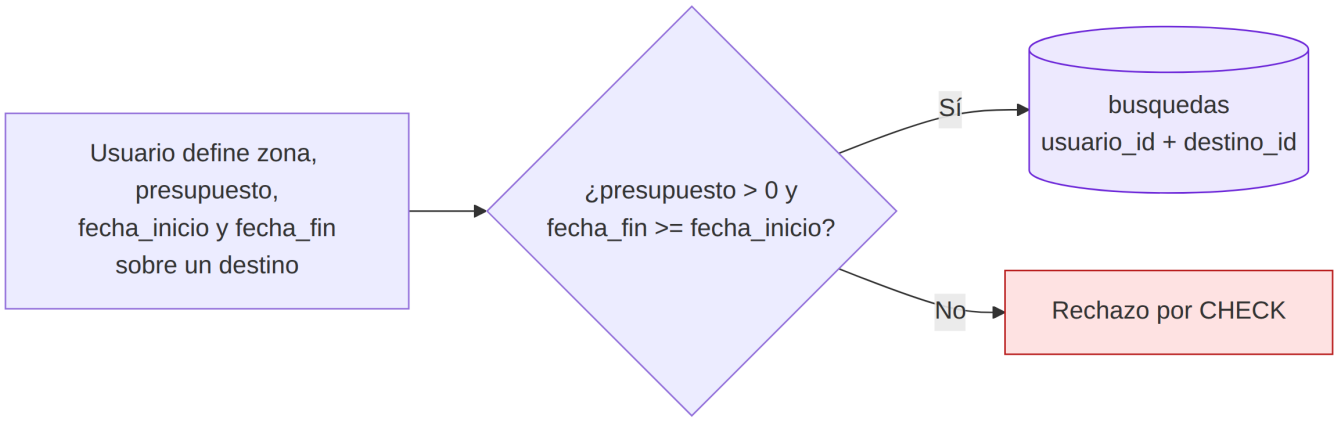


Figura 4 · Fase 2: creación de la búsqueda

3.3 Fase 3 · Registro del catálogo del destino

Cada hospedaje se liga a un destino. Luego recibe, como máximo, una evaluación de IA (UNIQUE) y puede recibir varias atracciones cercanas.

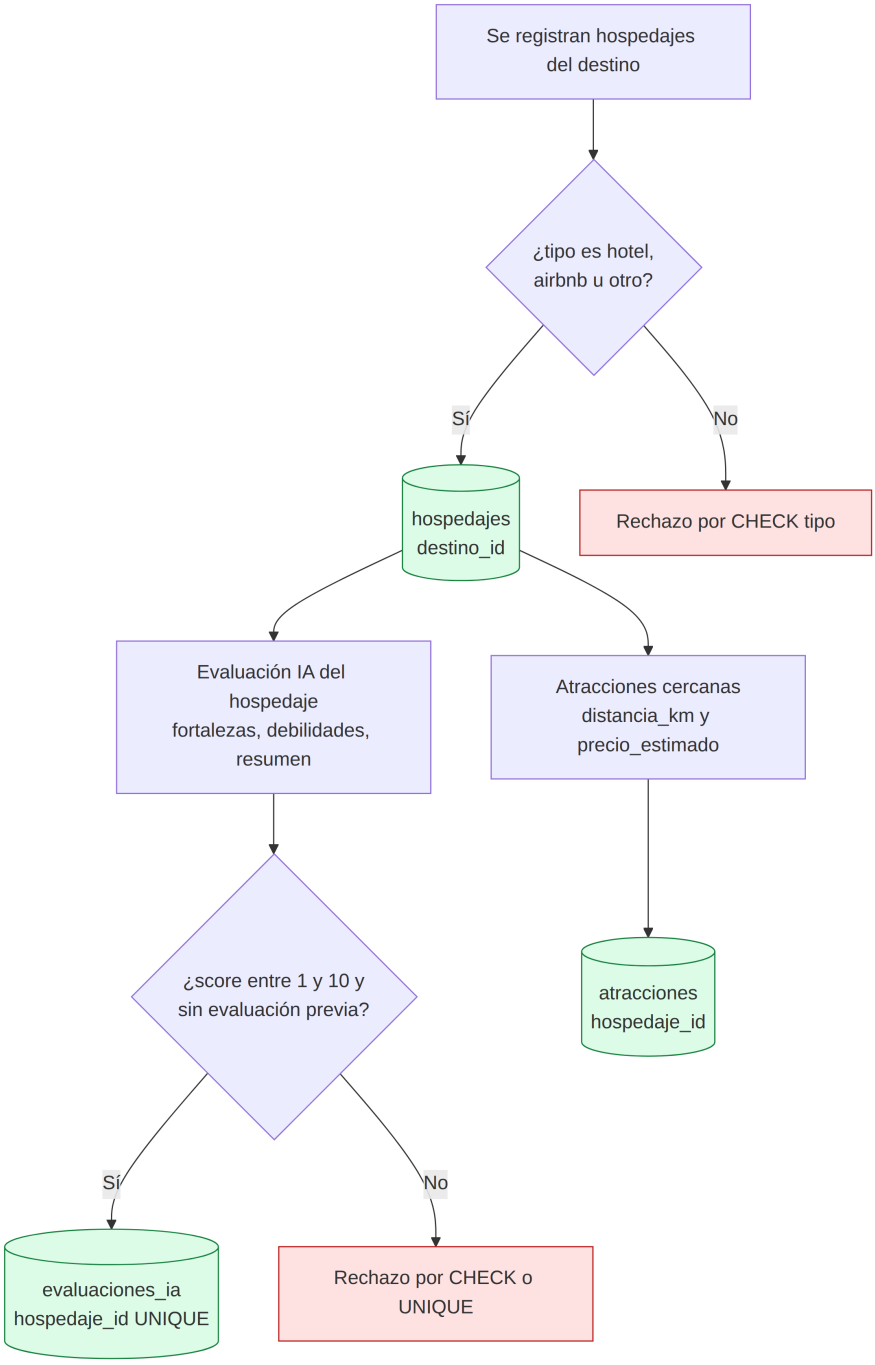


Figura 5 · Fase 3: hospedajes, evaluaciones de IA y atracciones

3.4 Fase 4 · Generación de recomendaciones (Top 3)

Se eligen hasta 3 hospedajes para la búsqueda. Cada uno ocupa una posición de 1 a 3, sin repetir posición ni hospedaje dentro de la misma búsqueda.

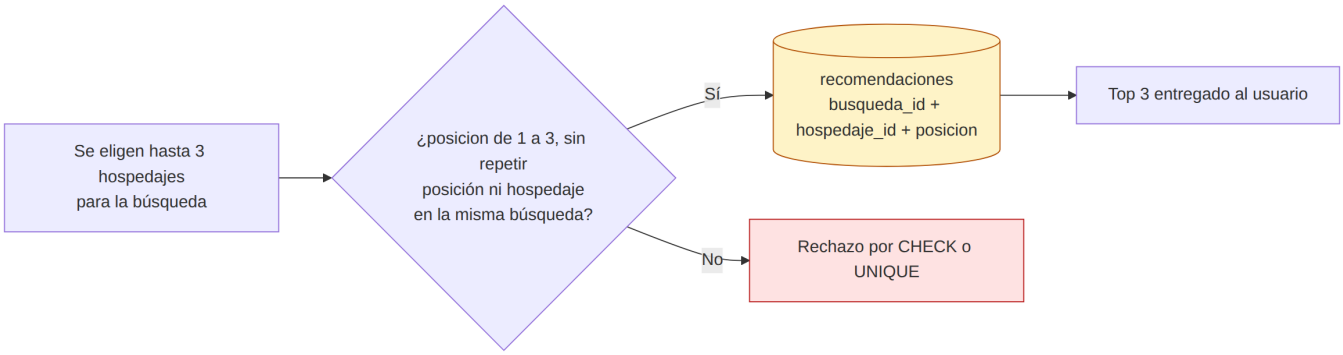


Figura 6 · Fase 4: recomendaciones

Anexo · Código Mermaid

Fuente de los diagramas, para pegarla en un README o en `mermaid.live`. El flujo de la sección 3 se muestra aquí como un solo diagrama.

A. Diagrama ER

```
erDiagram
    destinos ||--o{ hospedajes : "tiene"
    hospedajes |o--o| evaluaciones_ia : "es evaluado por"
    hospedajes |o--o{ atracciones : "tiene cerca"
    usuarios ||--o{ busquedas : "realiza"
    destinos ||--o{ busquedas : "es buscado en"
    busquedas ||--o{ recomendaciones : "genera"
    hospedajes ||--o{ recomendaciones : "es recomendado en"

    destinos {
        SERIAL id PK
        VARCHAR(100) ciudad "NOT NULL"
        VARCHAR(100) pais "NOT NULL"
        TIMESTAMP creado_en "DEFAULT CURRENT_TIMESTAMP"
    }

    hospedajes {
        INT id PK "IDENTITY"
        VARCHAR(200) nombre "NOT NULL"
        VARCHAR(20) tipo "CHECK hotel, airbnb, otro"
        DECIMAL_10_2 precio_noche
        DECIMAL_3_2 calificacion
        VARCHAR(200) direccion
        TEXT url_reserva
        INT destino_id FK "NOT NULL"
    }

    evaluaciones_ia {
        INT id PK "IDENTITY"
        TEXT puntos_fuertes
        TEXT puntos_debiles
        INT score_calidad_precio "CHECK 1 a 10"
        TEXT resumen_ejecutivo
        INT hospedaje_id FK, UK "nullable"
    }

    atracciones {
        INT id PK "IDENTITY"
        TEXT nombre
        TEXT categoria
        DECIMAL_4_2 distancia_km
        DECIMAL_10_2 precio_estimado "DEFAULT 0.00"
        INT hospedaje_id FK "nullable"
    }

    usuarios {
        INT id PK "IDENTITY"
        VARCHAR(50) nombre "NOT NULL"
        VARCHAR(50) apellido "NOT NULL"
        VARCHAR(60) correo UK "NOT NULL"
        VARCHAR(255) password_hash "NOT NULL"
    }

    busquedas {
        INT id PK "IDENTITY"
        INT usuario_id FK "NOT NULL"
        INT destino_id FK "NOT NULL"
        VARCHAR(100) zona "NOT NULL"
        DECIMAL_10_2 presupuesto "NOT NULL, CHECK > 0"
        DATE fecha_inicio "NOT NULL"
        DATE fecha_fin "NOT NULL, CHECK >= fecha_inicio"
    }

    recomendaciones {
        INT id PK "IDENTITY"
        INT busqueda_id FK "NOT NULL"
        INT hospedaje_id FK "NOT NULL"
        INT posicion "NOT NULL, CHECK 1 a 3"
    }
```

B. Diagrama de flujo (completo)

```
flowchart TD
    subgraph P1["1 · Registro de entidades raíz"]
        A1["Usuario se registra<br/>nombre, apellido, correo, password_hash"] --> A2{"¿Correo ya existe?<br/>UNIQUE correo"}
        A2 -- No --> A3[("usuarios")]
        A2 -- Sí --> A4["Rechazo por correo duplicado"]

        B1["Se identifica ciudad y país"] --> B2{"¿Existe ya?<br/>LOWER y TRIM de ciudad y país"}
        B2 -- No --> B3[("destinos")]
        B2 -- Sí --> B4["Se reutiliza el destino existente"]
    end

    A3 --> C1
    B3 --> C1
    B4 --> C1

    subgraph P2["2 · Búsqueda"]
        C1["Usuario define zona, presupuesto<br/>fecha_inicio y fecha_fin sobre un destino"] --> C2{"¿presupuesto > 0 y<br/>fecha_fin >= fec
        C2 -- Sí --> C3[("busquedas<br/>usuario_id + destino_id")]
        C2 -- No --> C4["Rechazo por CHECK"]
    end

    C3 --> D1

    subgraph P3["3 · Registro de catálogo del destino"]
        D1["Se registran hospedajes del destino"] --> D2{"¿tipo es hotel,<br/>airbnb u otro?"}
        D2 -- Sí --> D3[("hospedajes<br/>destino_id")]
        D2 -- No --> D4["Rechazo por CHECK tipo"]
        D3 --> D5["Evaluación IA del hospedaje<br/>fortalezas, debilidades, resumen"]
        D5 --> D6{"¿score entre 1 y 10<br/>y hospedaje sin evaluación previa?"}
        D6 -- Sí --> D7[("evaluaciones_ia<br/>hospedaje_id UNIQUE")]
        D6 -- No --> D8["Rechazo por CHECK o UNIQUE"]
        D3 --> D9["Atracciones cercanas<br/>distancia_km y precio_estimado"]
        D9 --> D10[("atracciones<br/>hospedaje_id")]
    end

    D7 --> E1
    D10 --> E1
    C3 --> E1

    subgraph P4["4 · Generación de recomendaciones Top 3"]
        E1["Se eligen hasta 3 hospedajes<br/>para la búsqueda"] --> E2{"¿posicion de 1 a 3,<br/>sin repetir posición ni hospedaje<br/>en la
        E2 -- Sí --> E3[("recomendaciones<br/>busqueda_id + hospedaje_id + posicion")]
        E2 -- No --> E4["Rechazo por CHECK o UNIQUE"]
        E3 --> E5["Top 3 entregado al usuario"]
    end
```